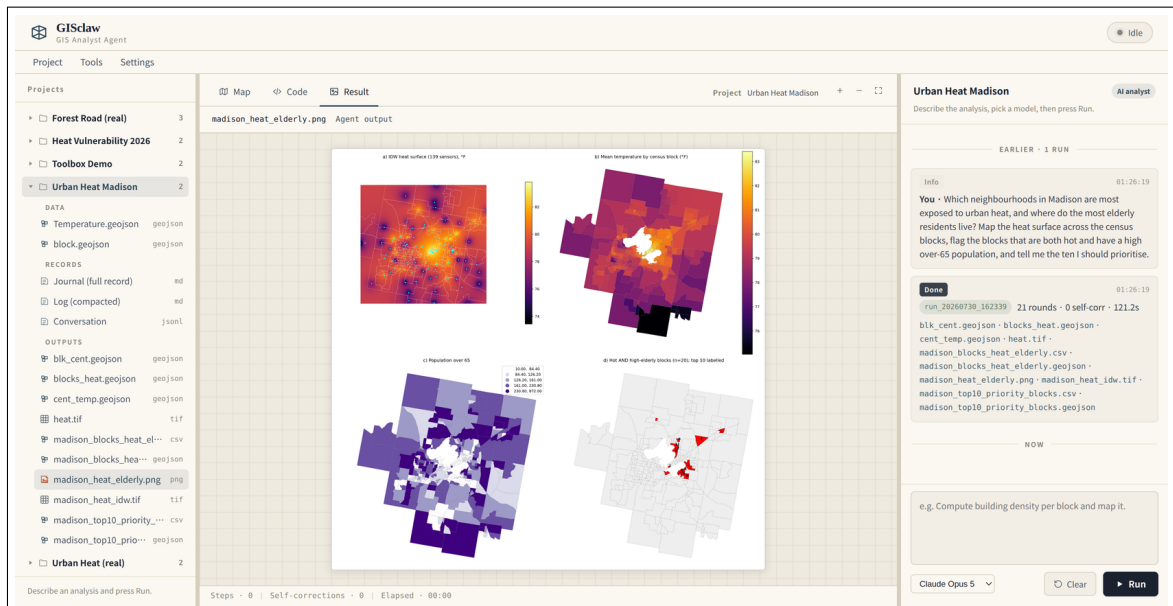


GISclaw

用户手册

说出你要做的空间分析,拿到做完的成果。



GISclaw 是一个地理空间分析自动化工具。你用平常说话的方式提出问题,它规划流程、在你自己的数据上执行,并交回图层、地图、表格,以及一份关于”它是如何做到的”的书面记录。本手册涵盖安装、日常使用、它留下的记录,以及让跨月项目保持可读的那套机制。

完全没有经验? 没用过 Docker、GitHub 或模型 API,请直接从 **附录 A 《从零开始安装》**(第 11 页)看起。那一节不预设任何基础:把环境装好、教你申请 key、一直带到跑完第一次分析,再回到这里。

Contents

1 GISclaw 做什么	3
1.1 常见的请求	3
2 安装	3
2.1 环境要求	3
2.2 步骤	3
3 日常使用	4
3.1 新建项目并挂载数据	4
3.2 怎么把需求说清楚	5
3.3 运行时看什么	5
4 一次运行受什么约束	5
5 每个项目留下什么	6
5.1 压缩日志	6
6 不同的输入,不同的反应	6
7 常驻偏好	7
8 Skills	7
9 算子工具箱	8
10 你的数据在哪	9
11 故障排查	9
12 与研究论文的关系	9
13 在依赖结果之前	9
14 许可与归属	10
A 从零开始安装	11
A.1 需要的三样东西	11
A.2 第 1 步 — 安装 Docker	11
A.2.1 Windows 10 或 11	11
A.2.2 macOS	11

A.2.3	Linux(Ubuntu、Debian 及衍生版)	11
A.2.4	验证装好了	12
A.2.5	接下来要敲的命令	12
A.3	第 2 步 — 取得 GISclaw 的文件	12
A.3.1	不用 git(最简单)	12
A.3.2	用 git	12
A.3.3	在这个文件夹里打开终端	13
A.4	第 3 步 — 申请 API key	13
A.5	第 4 步 — 启动 GISclaw	14
A.6	第 5 步 — 跑第一次分析	14
A.7	日常会用到的命令	14
A.8	装不起来时怎么办	15
A.9	怎么把花费控制住	15

1 GISclaw 做什么

空间分析大多是一串成熟操作的组合:重投影、插值、连接、统计、分级、出图。真正耗时间的是把它们按正确顺序拼起来,错误也藏在这里。

GISclaw 接受的输入,就是领域科学家平时说话的样子。封面那张四联图的全部输入是下面这一句:

Which neighbourhoods in Madison are most exposed to urban heat, and where do the most elderly residents live? Map the heat surface across the census blocks, flag the blocks that are both hot and have a high over-65 population, and tell me the ten I should prioritise.

据此,GISclaw 检查了两个图层,发现坐标系不一致,先做重投影;用 139 个观测点构建 IDW 热力面;对 269 个人口普查街区计算分区均值;把温度与 65 岁以上人口密度归一化后加权合并、排序;绘制图件;写出十个文件。共 21 步,耗时 121 秒。

1.1 常见的请求

你说	它产出
水位超过 2 米时,哪些路段会切断通往医院的通路?	淹没范围、受影响路段、被孤立的汇水区
规划道路 500 米范围内压占了多少保护林?	缓冲区、相交、按保护等级的受影响面积、影响图
哪些片区消防服务覆盖不足?	服务半径、覆盖空白、每个空白区内人口
对比这两个时相的林地变化并给出损失量。	变化栅格、增加/减少/净变化、发散配色地图

2 安装

2.1 环境要求

Docker 与 Docker Compose,约 3 GB 磁盘空间用于镜像,以及一家模型服务商的 API key。无需安装任何 GIS 软件:容器内已包含完整的开源地理空间栈。

没用过 Docker、GitHub 或模型 API? 下面四步是给已经具备这些条件的读者看的摘要。附录 A 从零讲一遍同样的内容——各操作系统怎么装 Docker、不用 git 怎么下载文件、各家服务商怎么申请 key,以及每一步卡住时该怎么办。

2.2 步骤

1. `git clone https://github.com/geumjin99/GISclaw.git`,进入 GISclaw
2. `docker compose up -d`
3. 打开 `http://localhost:8765`
4. **Settings** → **API keys...**,粘贴 key,点击 **Test**

Key 存在哪里。 Key 保存在服务端 `./projects/.gisclaw/settings.json`, 文件权限 600。它不会进入镜像、不会到达浏览器, 回传给界面的只有 `sk-abc...wxyz` 这样的掩码。若在 CI 或共享主机上使用, `.env` 中的环境变量可作为备用来源。

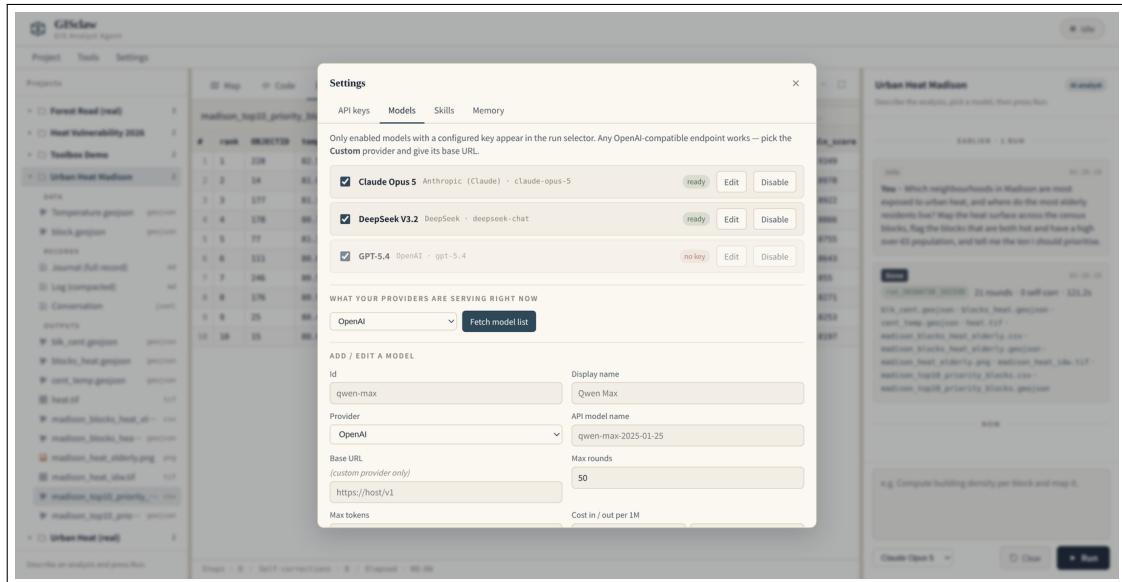


Figure 1: 模型注册表。列表实时从各服务商拉取, 新发布的模型无需等待本软件更新; 运行下拉里只出现确实配好 key 的模型。

3 日常使用

3.1 新建项目并挂载数据

项目就是一个工作文件夹。用 **Project** → **New project...** 新建, 再用 **Project** → **Add data...** 挂载数据 —— 它打开的是以挂载卷为根的服务端文件浏览器。也可以在左侧树里右键项目文件夹。

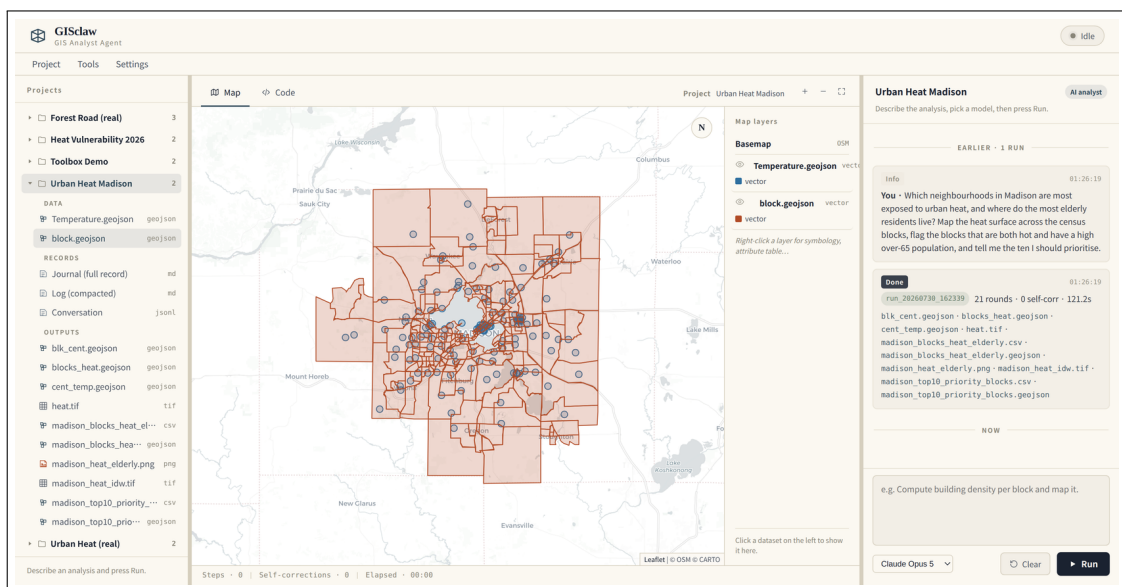


Figure 2: 图层渲染在地图上, 左侧是项目树, 右侧是对话。右键任意图层可改 符号化、查看属性表或缩放。

3.2 怎么把需求说清楚

说明目标、点出真正重要的约束、讲清要什么回来。字段名通常无需提供,agent 会自己读取 schema。

偏弱 分析一下热岛。

可用 用温度点插值,然后按街区统计。

好 把 TemperatureF 点插值成连续面,对每个人口普查街区计算分区均值, 把 blocks_meantemp.geojson 和一张 choropleth PNG 存到 pred_results/,并报告最热的五个街区以及有多少街区没有值。

要一个你能核对的数字——最热的五个街区、没有值的数量——是发现”跑完了但结果不对”最省事的办法。

3.3 运行时看什么

Thought、Action、Observation 实时流入对话。生成的代码出现在 **Code** 标签页,图件在 **Result**, 表格与文本在 **Table**,矢量或栅格产物自动渲染到地图。

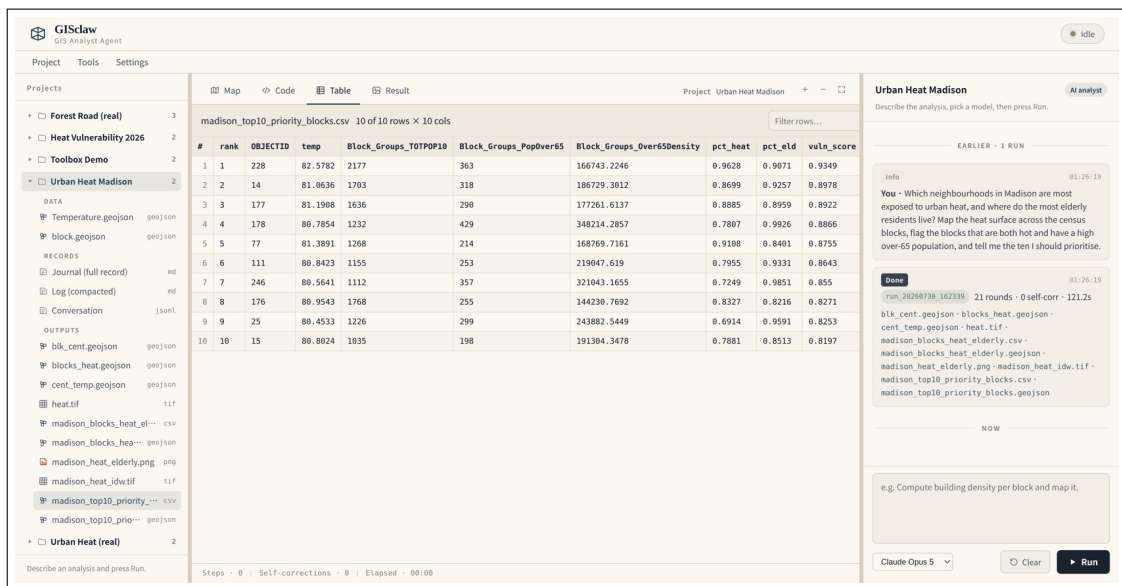


Figure 3: 提问、带产物清单的运行摘要,以及在查看器中打开的结果表。

4 一次运行受什么约束

GISclaw 在持久化 Python 沙箱上跑 ReAct 循环:看数据、推理下一步、写几行代码、读结果、继续。

其操作纪律来自本 agent 在真实多步 GIS 任务上约 1,800 次受控实验,并应用于每一次分析。

1. **先定完整路径再写代码。** 最主要的失败是”为错误的计划写了正确的代码”。
2. **列名从数据里读。** 绝不凭印象假设。
3. **把 CRS 当头等问题。** 投影混杂是常态;距离与面积运算在投影坐标系中进行,且每次连接后打印 null 率。
4. **优先使用确定性算子,而不是手写等价代码。**

5. **禁止静默填补缺失值。** 没有观测的单元保持 null。确需填补时,必须给出理由、数量与标记列,并在结论中写明。
6. **报错要诊断,不要重试。** 真正的错误在 traceback 末尾。
7. **结束前先核验。** 值域、计数,以及上图字段是否真的有变化。

第 5 条为什么存在。 若一次运行用另外 99 个单元的均值填满了 269 个中的 170 个,产出的地图看起来完整,而其中三分之二的格局是制造出来的 —— 并且能通过所有自动检查。明确说明只覆盖了 99 个单元,价值更高。

5 每个项目留下什么

projects/<你的项目>/	
├── data/	你挂进来的数据
├── outputs/	产出
├── JOURNAL.md	完整记录:每次运行、每一轮
├── LOG.md	压缩摘要,每次运行一条
├── chat.jsonl	对话,由界面重建
└── runs/run_<时间戳>/	
├── code.py	当时实际执行的代码
├── trace.jsonl	每一条 Thought / Action / Observation
└── pred_results/	这次的产物,永不被覆盖

刷新浏览器、重启容器、三个月后再来:对话都会从 chat.jsonl 恢复。点击任意历史运行 可回放推理过程并载入当时执行的代码。

该信哪一份。 outputs/ 是便于取用的汇总副本,后续运行可能以同名文件覆盖; runs/.../ pred_results/ 永不改动,任何一次具体结果都可从那里精确复现。

5.1 压缩日志

长期项目会堆出没人重读的记录。每次运行结束后,GISclaw 用一次模型调用把轨迹压成五个字段 —— Result、Method、Numbers、Caveats、Carries forward —— 追加进 LOG.md。注入下一次运行上下文的正是这份摘要。

6 不同的输入,不同的反应

GISclaw 会先判断收到的是哪一类消息,再决定怎么响应。因此它在一件工作的全过程都用得上,而不只是在你已经确知要跑什么的那一刻。

- **分析请求** —— 启动一次运行。
- **关于项目的提问** —— 做过什么、某个图层里有什么、某个值为什么看着不对 —— 用一次调用从记录里作答。
- **想先斟酌的方案** —— 可以讨论。你可以描述一个思路、问它面对现有数据会怎么做、让它比较两种方法的取舍,在任何计算发生之前把方案定下来。
- **日常对话** —— 就当日常对话处理。
- **超出范围的请求** —— 婉拒,并说明它能处理什么。

讨论一个方法只需一次短调用;真跑一遍可能要几分钟和一美元。先把路线谈拢,往往是通向正确答案更省的那条路。



Figure 4: 压缩日志。每一条都由模型依据该次运行的完整轨迹写成。

7 常驻偏好

全局 MEMORY.md 保存适用于一切工作的约定:配色方案、统一采用的分级方法、本地区的 投影坐标系、每张交付图必须包含的要素。在 **Settings** → **Memory...** 写一次,或在任意消息上悬停点击 **Remember**,它会注入每一次运行。

内容以规则为主。HTML 注释中的文字在注入前会被剥除,因此写给自己的说明不产生任何开销。

8 Skills

一个 skill 是用来固化”某一类分析该怎么做”的目录包:

```
<skill>/
├── SKILL.md          YAML frontmatter 与一段简短路由
├── manifest.yaml     可选的声明式加载计划
└── references/*.md   深度内容,仅在某一步需要时打开
```

加载是渐进的,因此技能库再大也负担得起:一行描述常驻上下文(约 80 token),命中时才载入 路由,reference 文件按需逐个打开。

层级	内容	何时加载
目录	名称与一行描述	始终
路由	SKILL.md 正文与文件清单	被打开或被匹配时
深度	单个 reference 文件	某一步需要时

应用内置两个 skill。gis-analysis-discipline 承载第 4 节的纪律,默认常驻;gis-workflow-recipes 收录四类任务的分步模板:插值加分区统计、多准则适宜性叠加、邻近与影响评估、两时相变化检测。

技能包支持以 .zip 或文件夹导入、在应用内编辑、导出分享。编写时请在 frontmatter 中声明 keywords:,服务端据此匹配并预加载合适的 skill。

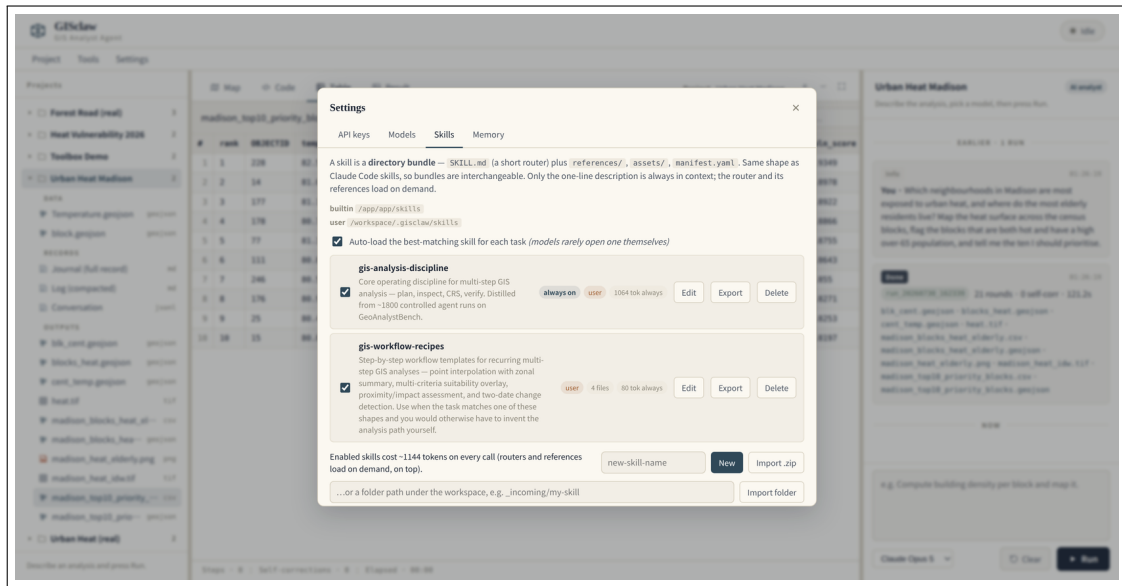


Figure 5: Skills 面板,显示来源、包内文件数,以及每个启用的 skill 增加的 token 开销。

9 算子工具箱

应用内置 28 个确定性的地理处理算子:重投影、缓冲、裁剪、相交、相减、合并、融合、空间与属性连接、分区统计、坡度、坡向、山体阴影、栅格化、IDW 等。

保留它们有两个实际理由。其一,它们经过测试且正确处理 CRS,agent 会优先调用而不是手写等价代码;其二,当你已经确知要做哪一步时,直接在面板里运行完全不产生 API 花费,结果立即上图。

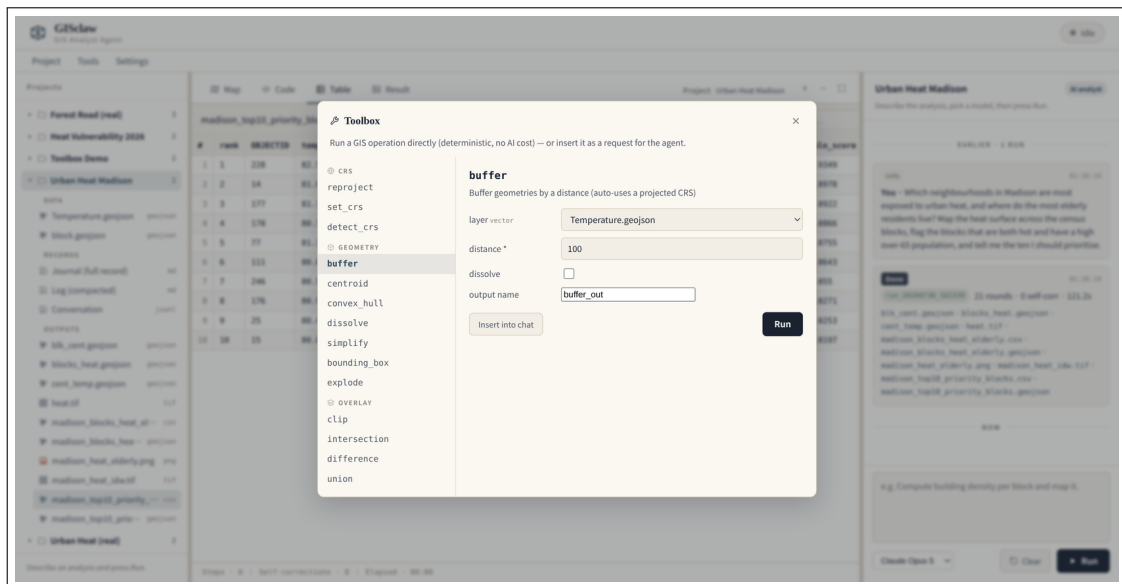


Figure 6: 每个算子都有真实的参数表单。Run 直接确定性执行; Insert into chat 把它作为更大流程中的一步交给 agent。

10 你的数据在哪

分析在容器内针对原生格式进行:Shapefile、GeoTIFF、GeoPackage、CSV、NetCDF。转成 GeoJSON 或 PNG 只发生在浏览器显示这一层,因为浏览器只能画这些。文件本身留在你的机器上。发送给模型 服务商的,是你的提问以及 agent 对数据的观察。

11 故障排查

现象	检查
模型下拉显示 No model configured	没有任何服务商配置了 key。进 Settings → API keys 并点 Test 。
运行刚开始就报 key 错误	所选模型的服务商没有 key;换一个标记为 ready 的模型。
栅格操作在宿主机上失败	栅格请在容器内运行;宿主机 PROJ 数据库损坏会影响 rasterio.warp,矢量不受影响。
更新后界面看起来还是旧的	强制刷新浏览器,并确认控制台报告的 app.js 版本符合预期。
outputs/ 里出现中间文件	每一步 geoprocess 都写入 pred_results/,中间产物会与最终成果一并复制。

服务端日志:docker logs -f gisclaw_app。单次运行日志: projects/<项目>/runs/<run_id>/run.log。

12 与研究论文的关系

arXiv:2603.26845 描述的系统是为受控实验构建的评测框架。本桌面应用在那套 agent 核心之上重建,此后已有大量分化:界面可用的算子工具箱、带日志与压缩摘要的项目持久化、具备渐进披露的 skill 包、全局偏好 memory,以及在线模型发现。

论文报告的实验结果由研究框架在其冻结 commit 上产生。复现工作应使用那份 artifact。

13 在依赖结果之前

GISclaw 的分析路径和代码都是它自己规划、自己写的,而这个规划来自大语言模型。它可能出错,而且可能在语气非常肯定的情况下出错。第 4 节那套操作纪律来自约 1800 次受控实验,能显著降低几类典型失败——连接后全为空、坐标系不一致、插值结果超出观测范围、缺失值被悄悄填补——但不能消除它们。

每次运行都会把实际执行的代码、完整轨迹和产物留在磁盘上,正是为了让你能核对。请核对。把产出当作供专业人员复核的草稿,而不是结论;在错误答案会造成危害的场合尤其要谨慎——洪涝与灾害制图、应急响应、基础设施选址、环境影响评价、土地权属、公共卫生。GISclaw 是研究工具,其产出不构成测绘、工程、环境、法律或规划方面的专业意见。

作者对模型产生的结果、以及据此做出的决定不承担任何责任,本软件不提供任何形式的担保。另请注意:分析在本地运行,但推理不在——对你数据的描述(文件名、字段名、坐标系、范围、统计摘要、出现在程序输出里的样本值)会发送给你所配置的模型服务商。完整条款见 DISCLAIMER.md。

14 许可与归属

Copyright © 2026 Han Jinzhen。GISclaw 是自由软件,采用 GNU Affero 通用公共 许可证第 3 版或更高版本。你可以免费用于任何用途,包括商业用途。如果你分发修改过的版本,或把修改过的版本作为网络服务提供给他人使用,AGPL 第 13 条要求你以同一许可向这些用户提供 你那份的完整源码。原样运行、或仅为自己内部使用而修改,不产生任何公开义务。

如需把 GISclaw 嵌入闭源产品或运营闭源的托管服务,可向著作权人获取单独的商业许可,见 `COMMERCIAL-LICENSE.md`。配套论文对应的研究代码仍以 MIT 许可保留在 `paper-v2` 分支上,以保证已发表结果可按论文所引用的条款复现。

内置示例数据来自 GeoAnalystBench(Zhang et al., 2025),依 Apache-2.0 分发,并有独立的引用要求;完整的第三方材料清单见 `examples/urban-heat-madison/README.md` 与 `THIRD_PARTY_NOTICES.md`。

A 从零开始安装

本附录假定你没用过 GitHub、没用过 Docker、也从没申请过 API key。按顺序做完,就能在自己的电脑上把 GISclaw 跑起来。第一次请预留一小时左右,其中大部分时间是在等下载,而不是在敲命令。

A.1 需要的三样东西

Docker

GISclaw 依赖的地理空间库 —— GDAL、GEOS、PROJ、GeoPandas、rasterio —— 以难装出名,而且版本不对时会悄悄弄坏坐标处理,报错还未必看得出来。Docker 绕开了这个问题:你下载的是一个已经装好、且验证可用的完整环境,它与你电脑上的其它软件互相隔离。

GISclaw 的文件

一个装着代码和配置的文件夹,发布在 GitHub 上。GitHub 只是这个文件夹的存放处;你不需要注册账号,也不需要懂版本控制就能下载。

一个 API key

地理空间计算由 GISclaw 自己完成,但推理来自服务商服务器上运行的大语言模型。key 是一串很长的密钥,用来标识你在该服务商的账号并按用量计费。GISclaw 本身没有账号、没有订阅,也不收授权费。

除此之外不需要别的。不需要 QGIS,不需要 ArcGIS,也不需要预先装 Python。

A.2 第 1 步 —— 安装 Docker

Windows 10 或 11

1. 从 <https://www.docker.com/products/docker-desktop/> 下载 **Docker Desktop**,选择与你 CPU 匹配的版本(绝大多数情况是 **AMD64**; 只有骁龙芯片的机器才选 ARM64)。
2. 运行安装程序,保持 **Use WSL 2 instead of Hyper-V** 处于勾选状态。
3. 如果提示重启,就重启。
4. 启动 Docker Desktop,等到鲸鱼图标不再动、状态显示 Engine running。

最常见的拦路虎是主板 BIOS/UEFI 里关掉了硬件虚拟化。Docker Desktop 会明确提示;该选项按主板不同叫 Intel VT-x、AMD-V 或 SVM Mode。

macOS

在同一地址下载,注意选对芯片版本:M 系列机器选 **Apple Silicon**,老机器选 **Intel**。选错了装上也起不来。不确定的话看**苹果菜单** → **关于本机**。把应用拖进”应用程序”,启动,系统询问时同意安装特权助手。

Linux(Ubuntu、Debian 及衍生版)

多数发行版自带的 docker.io 版本太旧,不支持本文用的 docker compose 语法。请按 <https://docs.docker.com/engine/install/> 从 Docker 官方源安装,然后把自己加进 docker 组,免得每条命令都要 sudo:

```
| sudo usermod -aG docker $USER
```

需要注销后重新登录才会生效。

验证装好了

打开终端,依次运行:

```
docker --version
docker compose version
docker run --rm hello-world
```

第三条会下载一个很小的测试镜像并打印 Hello from Docker!。能打印出来,说明安装没问题,本附录后面的步骤都能跑通。

带连字符的 `docker-compose` 是已经淘汰的 v1。 本手册用的是中间带空格的 `docker compose`。如果你机器上只有带连字符的那个,请先升级再继续 —— v1 读不懂本项目的 配置文件。

接下来要敲的命令

Docker 装好之后,剩下的就是三条命令加一个浏览器标签页。第 2 到第 4 步会逐行解释、并说明某一步不对劲时怎么办;这里先把全貌摆出来,让你知道要去哪:

```
git clone https://github.com/geumjin99/GISclaw.git
cd GISclaw
docker compose up -d
```

然后在浏览器打开 <http://localhost:8765>,在 **Settings** → **API keys...** 里粘贴 API key。整个安装就这些。

A.3 第 2 步 —— 取得 GISclaw 的文件

下面两种方式都行,都不需要 GitHub 账号。

不用 git(最简单)

1. 打开 <https://github.com/geumjin99/GISclaw>。
2. 点绿色的 **Code** 按钮,选 **Download ZIP**。
3. 解压到一个你找得到的地方,放”文档”里就行。会得到一个名为 GISclaw-main 的文件夹。

尽量选一个路径里没有空格、没有中文的位置。带空格或中文通常也没问题,但万一后面出怪毛病,这是第一个该排除的因素。

用 git

```
git clone https://github.com/geumjin99/GISclaw.git
cd GISclaw
```

好处是以后 `git pull` 一下就能更新,不用整包重下。

在这个文件夹里打开终端

从这里开始的每条命令,都必须在包含 `docker-compose.yml` 的那个文件夹里运行。

- **Windows** — 在文件资源管理器里打开该文件夹,点一下地址栏,输入 `cmd` 回车。或者在文件夹空白处右键,选**在终端中打开**。
- **macOS** — 右键该文件夹,选**服务** → **新建位于文件夹位置的 终端窗口**。如果没有这一项,去**系统设置** → **键盘** → **键盘快捷键** → **服务**里启用。也可以打开”终端”,输入 `cd` 之后把文件夹拖进窗口。
- **Linux** — 在文件夹里右键,选**在终端中打开**。

用 `ls`(macOS/Linux)或 `dir`(Windows)列一下,能看到 `docker-compose.yml` 就说明位置对了。

A.4 第 3 步 — 申请 API key

费用直接付给模型服务商。花销不大:一次分析大致在不到一分钱到几毛钱之间,取决于用哪个模型、任务跑多久。

下面任选一家即可开始:

- **DeepSeek** — <https://platform.deepseek.com>
便宜得多,用来熟悉软件完全够用,国内访问也方便。推荐作为第一选择。
- **Anthropic(Claude)** — <https://console.anthropic.com/settings/keys>
Claude Opus 5 是 GISclaw 的默认模型,处理长链条多步分析最强。
- **OpenAI** — <https://platform.openai.com/api-keys>
用的人最多,文档齐全。
- **Google Gemini** — <https://aistudio.google.com/apikey>
有免费额度,适合零成本先试一次。

各家流程基本一致:

1. 在上面的控制台地址注册账号。
2. 绑定支付方式。多数服务商要求账户里先有余额,第一次调用才会成功;有些会送少量试用额度。
3. 找到 **API keys** 或 **Credentials** 那一栏。
4. 新建一个 key,起个认得出的名字,比如 GISclaw。
5. **立刻复制**。完整的 key 只会显示这一次。弄丢了就把它删掉重新建一个 — 没有再看一遍的办法。

把 key 当银行卡对待。 拿到它的人就能花你的余额。绝对不要粘进聊天消息、截图、问题反馈,或者提交到代码仓库的文件里。一旦泄露,立刻去服务商控制台吊销并换新的。GISclaw 自己把 key 存在服务端的 `./projects/.gisclaw/settings.json`,文件权限 600,回传给浏览器的永远只是 `sk-abc...wxyz` 这样的掩码。

各家控制台改版频繁。如果上面写的入口位置对不上,就找 **API keys**、**Credentials** 或 **Billing** 这类字样。

A.5 第 4 步 — 启动 GISclaw

在第 2 步打开的终端里:

```
| docker compose up -d
```

第一次运行要现场构建镜像,很慢。 Docker 会从 conda-forge 拉取地理空间栈并安装 Python 包,最终约 1.8 GB。普通网络下要十到三十分钟,中间会有很长时间看着像卡住了 —— 那是正常的,它在下载。之后每次启动只需几秒,因为构建好的镜像会被复用。

国内网络。 如果拉取镜像或 conda-forge 包长时间没有进展甚至超时,通常不是本软件的问题,而是到 Docker Hub 与 conda-forge 的连接。可以在 Docker Desktop 的 **Settings** → **Docker Engine** 里配置镜像加速地址,或在能稳定连通的网路下完成这一次构建 —— 构建只需成功一次,之后都用本地镜像。

命令返回后,在浏览器打开 <http://localhost:8765>。

进 **Settings** → **API keys...**,把第 3 步的 key 粘到对应服务商那一栏,点 **Test**。测试通过后,该服务商的模型才会出现在运行下拉框里。如果你发现模型列表是空的,原因也在这里:只有配了可用 key 的模型才会被列出。

A.6 第 5 步 — 跑第一次分析

软件自带一个示例。应用只能看到挂载工作区里的文件,也就是与 docker-compose.yml 同级的 projects 文件夹,所以先把示例复制进去:

```
| cp -r examples/urban-heat-madison projects/
```

Windows 下在同一个文件夹里执行:

```
| xcopy /E /I examples\urban-heat-madison projects\urban-heat-madison
```

然后在浏览器里:**Project** → **New project...** 建项目并命名, **Project** → **Add data...** 选中那两个 GeoJSON 文件,再输入一句需求,例如按人口普查街区制作夏季高温暴露图,并告诉我哪些街区最严重。右侧会实时显示它的思考与操作过程。

A.7 日常会用到的命令

都在放着 docker-compose.yml 的那个文件夹里执行。

命令	作用
docker compose up -d	后台启动 GISclaw
docker compose stop	停止,保留容器与镜像
docker compose down	停止并删除容器;projects/ 目录不受影响
docker compose logs -f	实时查看服务端日志,Ctrl+C 退出查看
docker compose restart	重启,改完设置文件后用这个
docker compose up -d --build	更新源码后重新构建镜像

你的成果从来不存在容器里。项目、数据、产物、日志和 key 全在你自己磁盘上的 projects 文件夹中,所以删掉容器再重建不会丢任何东西。

A.8 装不起来时怎么办

下表针对启动阶段的问题。分析过程中的问题请看前面的”故障排查”一节。

你看到的	怎么办
docker: command not found	Docker 没装,或者不在 PATH 里。重装一次;Windows 和 macOS 上还要确认 Docker Desktop 确实在运行。
Cannot connect to the Docker daemon	引擎没启动。启动 Docker Desktop,Linux 上执行 <code>sudo systemctl start docker</code> 。
docker.sock 的 permission denied(Linux)	你的用户不在 docker 组里。执行 <code>sudo usermod -aG docker \$USER</code> ,然后注销重新登录。
env file .env not found	你的 Docker Compose 版本低于 2.24。升级它,或者在同一文件夹里建一个空的 .env 文件。
port is already allocated	8765 端口被别的程序占了。关掉那个程序,或者把 docker-compose.yml 里改成 "8766:8765",改用 <code>http://localhost:8766</code> 。
构建到一半下载失败	基本都是网络问题。重新执行 <code>docker compose up -d</code> —— 已完成的层有缓存,会接着下,不会从头再来。
页面打不开	容器可能已经退出。执行 <code>docker compose logs</code> ,看最后几行。
模型列表是空的	还没有 key 通过 Test 。回到第 3 步。
界面像是旧版	强制刷新:Ctrl+Shift+R,macOS 上是 Cmd+Shift+R。

A.9 怎么把花费控制住

- 熟悉软件的阶段先用 DeepSeek;等到这次分析真的重要了,再换更强的模型。
- 一句说清楚的需求,比几句含糊的便宜 —— agent 不必花好几轮去猜你到底想要什么。
- 复用已有产物。在之前做好的图层上加一列,远比重算一遍便宜;GISclaw 会被告知此前的产物,正是为了让它能这么做。
- 在服务商控制台设一个每月消费上限。每家都提供这个功能,而且这是唯一能硬性兜底的手段。
- 每次运行都会记录自己的花费,Project → Log (compacted) 与 Journal 里都能看到。